DevOps Interview Preparation Series(Day 5,6)

by T Keerti

1)What is Terraform?

Terraform is an open-source Infrastructure as Code (IaC) tool created by HashiCorp that allows you to define, provision, and manage cloud infrastructure (AWS, Azure, GCP, etc.) using declarative configuration files written in HCL (HashiCorp Configuration Language)

```
resource "aws_instance" "web" {
  ami = "ami-0123456789"
 instance_type = "t2.micro"
}
```

Here, you are **declaring what** you want (not how).
Terraform figures out how to reach that state.

2)Why Terraform?

Feature	Description
IaC (Infrastructure as Code)	Manage infra as code — track, review, and version it.
Multi-cloud support	Works across AWS, Azure, GCP, etc.
Declarative language (HCL)	You define desired state, Terraform does the rest.
Execution plan	`terraform plan` shows changes before applying.
Dependency graph	Terraform auto-resolves dependencies between resources.
Idempotent	Re-running doesn’t recreate unchanged resources.
State management	Tracks infra in `.tfstate` file.
Modules & Reusability	Create reusable, parameterized infrastructure blocks.
Immutable infrastructure	Destroy + recreate pattern ensures clean environments.
Collaboration	Remote state + locking avoids drift & conflicts.

3)How Terraform Works (Architecture)
.tf files → Terraform Core → Providers → Cloud APIs (AWS, Azure, etc.)
Components:

1. **Configuration files (.tf):**
   - Define what to create (e.g., EC2, VPC).
2. **Terraform Core:**
   - Handles state, plan, and apply logic.
3. **Providers:**
   - Plugins that translate Terraform code into API calls.
4. **State file (.tfstate):**
   - Stores current infra snapshot.
5. **Backend:**
   - Defines where state file is stored (local/S3/DynamoDB).
6. **Provisioners:**
   - Run scripts/commands after resource creation.

4)Terraform Architecture – Step-by-Step Flow
Write Code

Define desired infrastructure in `.tf` files.

Initialize

```
terraform init(Downloads providers & configures backend.)
```

Plan
terraform plan(Compares desired vs actual infrastructure.)
Apply

```
terraform apply(Creates/updates/destroys resources)
```

State Update(Saves real infrastructure info to `terraform.tfstate`).

Destroy(Deletes resources defined in `.tf`.)

5)**T**erraform Lifecycle Phases

> What’s the difference between `plan` and `apply`?
> A: `plan` shows what changes Terraform *would* make, `apply` actually makes them.

6) INSTALLATION & SETUP
Installing Terraform
Linux:

```
sudo apt update
sudo apt install unzip -y
wget https://releases.hashicorp.com/terraform/1.8.0/terraform_1.8.0_linux_amd64.zip
unzip terraform_1.8.0_linux_amd64.zip
sudo mv terraform /usr/local/bin/
terraform -version
```

Windows:
Download ZIP → Extract → Add to `PATH`
Verify: `terraform -v`

AWS CLI Setup

```
sudo apt install awscli -y
aws configure
Fill:
AWS Access Key ID: <access-key>
AWS Secret Access Key: <secret>
Default region: ap-south-1
```

7) IAM Setup for Terraform

1. Go to **IAM → Users → Create User**
2. Add: `terraform-user`
3. Permissions:
   - AdministratorAccess (for learning)
   - Or: EC2, S3, VPC, IAM access limited policies.
4. Create **Access key + Secret key**
5. Configure via `aws configure`

8)What are best practices for credentials in Terraform?

Never hardcode keys in `.tf`
Use environment variables or IAM roles
Use Vault or AWS Secrets Manager for secrets

9)TERRAFORM BASICS
Provider(Downloads plugins for the cloud.)

```
provider "aws" {
 region = "ap-south-1"
}
```

Resource(Creates AWS EC2)

```
resource "aws_instance" "web" {
 ami          = "ami-0abcdef"
 instance_type = "t2.micro"
}
```

Data Source
Used to fetch **existing** resources.

```
data "aws_ami" "ubuntu" {
 most_recent = true
 owners     = ["099720109477"]
 filter {
   name   = "name"
   values = ["ubuntu/images/hvm-ssd/ubuntu-focal*"]
 }
}
```

Difference between `resource` and `data`?(`resource` creates infra; `data` reads existing infra.)

10)Variables

```
variable "instance_type" {
  default = "t2.micro"
}
```

Usage:
instance_type = var.instance_type

Precedence:

1. CLI `-var`
2. `.tfvars`
3. Environment variable
4. Default

Outputs(Used to expose resource attributes for use in modules or pipeline)

```
output "instance_ip" {
 value = aws_instance.web.public_ip
}
```

Locals(Used for reusing computed values inside a file.)

```
locals {
 tags = {
   Environment = "Dev"
   Owner      = "Keerti"
 }
}
```

11)What is State(JSON file that records the mapping between **real infrastructure** and Terraform resources)

12)Why state is important

Without state, Terraform wouldn’t know:
  What exists
  What needs changes
  Dependencies

13)State Storage Options

Local: `terraform.tfstate`

Remote

```
terraform {
 backend "s3" {
   bucket = "terraform-state-bucket"
   key    = "env/dev/terraform.tfstate"
   region = "ap-south-1"
 }
}
```

Add DynamoDB for state locking:

```
dynamodb_table = "terraform-lock"
```

14)Desired vs Actual vs State

```
 Type   | Meaning
---------- ----------------------
 DesiredW - That’s defined in code
 Actual - What’s live in AWS
 state -Terraform’s record
```

15)**Count**

```
resource "aws_instance" "web" {
 count = 3
 ami = "ami-0abcdef"
}
```

for_each

```
resource "aws_security_group_rule" "rule" {
 for_each = toset(["22", "80"])
 from_port = each.value
 to_port   = each.value
}
```

Conditional

```
count = var.env == "prod" ? 3 : 1
```

Dynamic Block(Dynamic blocks reduce repetitive code — used in SG, tags, etc)

```
dynamic "ingress" {
 for_each = var.ingress_rules
 content {
   from_port   = ingress.value.port
   to_port     = ingress.value.port
   protocol    = "tcp"
   cidr_blocks = ["0.0.0.0/0"]
 }
}
```

16)What is a VPC?

VPC (Virtual Private Cloud) is your private isolated network in AWS.
It allows you to define:

- **CIDR range** (e.g., 10.0.0.0/16)
- **Subnets** (public & private)
- **Routing tables**
- **Internet Gateways / NAT Gateways**
- **Security Groups / NACLs**

17)CIDR (Classless Inter-Domain Routing)

- Defines **range of IPs** your network will have.
  Example:
  `10.0.0.0/16` → gives ~65,536 IPs
  `10.0.1.0/24` → gives 256 IPs
  How do you decide CIDR range for a VPC?
  A: Depends on number of subnets, environment separation, and IP allocation strategy.

18)Terraform VPC Example

```
resource "aws_vpc" "main" {
 cidr_block          = "10.0.0.0/16"
 enable_dns_support   = true
 enable_dns_hostnames = true

 tags = {
   Name = "main-vpc"
 }
}
```

19)Internet Gateway (IGW)
To allow internet access for public subnets.

```
resource "aws_internet_gateway" "igw" {
 vpc_id = aws_vpc.main.id
 tags = { Name = "main-igw" }
}
```

20)Subnets

Public Subnet

```
resource "aws_subnet" "public" {
 vpc_id                = aws_vpc.main.id
 cidr_block            = "10.0.1.0/24"
 map_public_ip_on_launch = true
 availability_zone      = "ap-south-1a"

 tags = { Name = "public-subnet" }
}
```

Private Subnet

```
resource "aws_subnet" "private" {
 vpc_id          = aws_vpc.main.id
 cidr_block       = "10.0.2.0/24"
 availability_zone = "ap-south-1b"

 tags = { Name = "private-subnet" }
}
```

Route Table + Association

```
resource "aws_route_table" "public" {
 vpc_id = aws_vpc.main.id
 route {
   cidr_block = "0.0.0.0/0"
   gateway_id = aws_internet_gateway.igw.id
 }
 tags = { Name = "public-rt" }
}

resource "aws_route_table_association" "public_assoc" {
 subnet_id      = aws_subnet.public.id
 route_table_id = aws_route_table.public.id
}
```

NAT Gateway (For private subnets to access the Internet)

```
resource "aws_eip" "nat_eip" {
 vpc = true
}

resource "aws_nat_gateway" "nat" {
 allocation_id = aws_eip.nat_eip.id
 subnet_id     = aws_subnet.public.id
}
```

Then add a **private route table**:

```
resource "aws_route_table" "private" {
 vpc_id = aws_vpc.main.id
 route {
   cidr_block     = "0.0.0.0/0"
   nat_gateway_id = aws_nat_gateway.nat.id
 }
}
```

21)VPC Peering (Connect two VPCs)

```
resource "aws_vpc_peering_connection" "peer" {
 vpc_id      = aws_vpc.main.id
 peer_vpc_id = aws_vpc.secondary.id
 auto_accept = true
}
```

> Why NAT instead of IGW for private subnet?
> A: NAT allows **outbound only** traffic from private instances — IGW would make them public.

22)EC2 MODULE DEVELOPMENT
Folder Structure

```
modules/
 ec2/
   main.tf
   variables.tf
   outputs.tf
main.tf
```

23)EC2 Module Code
main.tf

```
resource "aws_instance" "ec2" {
 ami          = var.ami
 instance_type = var.instance_type
 subnet_id     = var.subnet_id
 vpc_security_group_ids = [var.sg_id]
 key_name      = var.key_name

 tags = merge(
   var.common_tags,
   { Name = var.instance_name }
 )
}
```

variables.tf

```
variable "ami" {}
variable "instance_type" {}
variable "subnet_id" {}
variable "sg_id" {}
variable "key_name" {}
variable "instance_name" {}
variable "common_tags" { type = map(string) }
```

outputs.tf

```
output "instance_id" {
 value = aws_instance.ec2.id
}
```

Module Usage

```
module "frontend" {
 source        = "./modules/ec2"
 ami           = data.aws_ami.ubuntu.id
 instance_type  = "t3.micro"
 subnet_id      = aws_subnet.public.id
 sg_id          = aws_security_group.web_sg.id
 key_name       = "keerti-key"
 instance_name  = "frontend-ec2"
 common_tags    = local.tags
}
```

Why use modules?
A: To promote reusability, standardization, and easy management across multiple environments.

24)**LOAD BALANCER, TARGET GROUP, AND AUTOSCALING**

Application Load Balancer (ALB)

```
resource "aws_lb" "app_alb" {
 name             = "app-alb"
 internal          = false
 load_balancer_type = "application"
 security_groups    = [aws_security_group.alb_sg.id]
 subnets           = [aws_subnet.public.id]
}
```

Target Group

```
resource "aws_lb_target_group" "tg" {
 name     = "app-tg"
 port     = 80
 protocol = "HTTP"
 vpc_id   = aws_vpc.main.id
 health_check {
   path              = "/"
   healthy_threshold   = 3
   unhealthy_threshold = 2
 }
}
```

Listener

```
resource "aws_lb_listener" "listener" {
 load_balancer_arn = aws_lb.app_alb.arn
 port            = 80
 protocol         = "HTTP"

 default_action {
   type           = "forward"
   target_group_arn = aws_lb_target_group.tg.arn
 }
}
```

Launch Template + Auto Scaling

```
resource "aws_launch_template" "app" {
 name_prefix   = "app-template"
 image_id      = data.aws_ami.ubuntu.id
 instance_type = "t2.micro"
}



resource "aws_autoscaling_group" "asg" {
 desired_capacity     = 2
 max_size            = 3
 min_size            = 1
 launch_template {
   id      = aws_launch_template.app.id
   version = "$Latest"
 }
 vpc_zone_identifier = [aws_subnet.public.id]
 target_group_arns   = [aws_lb_target_group.tg.arn]
}
```

> How does ALB integrate with ASG?
> A: Target group automatically registers new EC2s from ASG.

25)**TERRAFORM LIFECYCLE, IMPORT, TAINT**

Lifecycle

```
lifecycle {
 create_before_destroy = true
 ignore_changes        = [tags]
}
```

Import Existing Resource

```
terraform import aws_instance.web i-0abcd1234
```

Taint Resource

```
terraform taint aws_instance.web
```

> When would you use `taint`?
> A: When a resource is corrupted or needs forced re-creation.

26)SECURING STATE & PIPELINES

Store state in **S3 backend**

- Lock state with **DynamoDB**
- Encrypt bucket with **SSE**
- Restrict bucket access via IAM policies

How do you handle Terraform in CI/CD?
A: Use Terraform Cloud or store state in remote backend (S3), run `init/plan/apply` in GitHub Actions or GitLab CI with role-based auth.

27)**EKS (KUBERNETES CLUSTER)**

Terraform creates:

- EKS Cluster
- Node Group
- IAM roles
- OIDC provider

```
module "eks" {
 source         = "terraform-aws-modules/eks/aws"
 cluster_name    = "demo-cluster"
 cluster_version = "1.29"
 subnets        = [aws_subnet.private.id]
 vpc_id         = aws_vpc.main.id
}
```

How does Terraform create EKS?(Via AWS EKS module, which provisions cluster + node groups.)

Question	Answer
How does Terraform create EKS?	Via AWS EKS module, which provisions cluster + node groups.
How do you connect EC2 to EKS?	Using same VPC and security groups.
How do you handle secrets in EKS Terraform?	Store in SSM Parameter Store or Secrets Manager, fetched via data block.

COMPLETE ARCHITECTURE FLOW

```
User → Route53 → CloudFront (CDN)
   ↓
  ALB (HTTPS Listener)
   ↓
 EKS Cluster (Pods)

Private RDS / SSM / Secrets Manager
   ↓
Monitoring (CloudWatch + Prometheus + Grafana)
   ↓
Terraform (Infra as Code)
   ↓
GitHub Actions / GitLab CI → Plan → Apply → Deploy
```

Scenario	Explanation
State Drift	Actual infra changed outside Terraform → fix with `terraform refresh`.
Team Collaboration	Use S3 backend + DynamoDB lock.
Blue-Green Deployments	Maintain two ALBs or ASGs — shift traffic gradually.
Canary Deployment (EKS)	Use weighted routing in ALB listener rules.
Terraform in Pipelines	Use role-based auth; never store keys; run plan + apply in stages.
Rollback	Use version-controlled `.tf` + previous state.
Workspaces	Separate logical environments (`dev`, `stage`, `prod`).
Provisioners	Use `remote-exec` to bootstrap EC2 post-creation.

Q	A
1️⃣ What happens during `terraform init`?	Initializes backend, downloads providers, and sets up workspace.
2️⃣ How does Terraform handle dependencies?	Automatically builds a dependency graph using resource references.
3️⃣ How can you reuse infrastructure?	Using **modules** with parameterized variables.
4️⃣ How do you manage sensitive data?	Use `sensitive = true`, SSM, Secrets Manager.
5️⃣ What if two people run `terraform apply`?	Locking prevents conflicts (via DynamoDB or Terraform Cloud).
6️⃣ How do you upgrade a module?	Change `version` in module source; run `init -upgrade`.
7️⃣ Can Terraform manage existing infra?	Yes, use `terraform import`.
8️⃣ What’s difference between Terraform & CloudFormation?	Terraform is multi-cloud + declarative; CloudFormation is AWS-only.
9️⃣ What’s the difference between local & remote exec?	Local runs on Terraform host; remote runs on target resource.
🔟 How do you handle drift detection?	`terraform plan` highlights drift automatically.

HOW TO WRITE A TERRAFORM RESOURCE BLOCK (Step-by-Step)

Understand the Structure

Every Terraform resource block follows this syntax:

```
resource "<PROVIDER>_<RESOURCE_TYPE>" "<LOCAL_NAME>" {
  # arguments go here
}
```

Example

```
resource "aws_instance" "web" {
  ami           = "ami-0abcdef"
  instance_type = "t2.micro"
}
```

Let's break it down:

```
| Part           | Meaning                                        |
| --------------- | ------------------------------------------------ |
| `resource`      | Keyword to define a resource block              |
| `"aws_instance"` | Type of resource from provider (AWS EC2 instance) |
| `"web"`         | Logical name — unique label used in references    |
| `{ ... }`       | Configuration arguments (key-value pairs)        |
```

Go to the Terraform Registry

This is the **official and most accurate** source for every resource, data source, and argument

STEP 3: Select Your Provider

Let's say you're using AWS.
Search for **"AWS provider"** or go directly:
Here, you'll see:

- Provider version
- Documentation for all AWS resources (EC2, S3, VPC, etc.)
- Input arguments and output attributes

Choose a Specific Resource

Example: we want to create an **EC2 instance**.

Click:
📁 `Resources → aws_instance`

Read Documentation Sections Carefully

Terraform Registry documentation always has **these key parts:**

```
| Section               | Purpose                                     |
| ----------------------- | -------------------------------------------- |
|  Example Usage.        | Basic ready-to-use code snippet              |
|  Argument Reference.   | Lists all inputs (Required + Optional)       |
|  Attributes Reference.  | Lists all outputs (values you can read later) |
|  Import.               | Syntax to import existing resources          |
|  Timeouts / Notes.     | Additional configuration info                |
```

Understand "Required" vs "Optional" Fields

Example from `aws_instance` docs:

Argument	Type	Required/Optional	Description
`ami`	string	✅ **Required**	The ID of the AMI to use.
`instance_type`	string	✅ **Required**	Type of instance (e.g., t2.micro).
`subnet_id`	string	Optional	The subnet ID in which to launch.
`vpc_security_group_ids`	list	Optional	Security group IDs.
`key_name`	string	Optional	SSH key to connect.
`tags`	map	Optional	Tags for the resource.

Build Your Resource Block

Now, based on docs, you can create a minimal or full block.

Minimal example (required only):

```
resource "aws_instance" "web" {
  ami           = "ami-0abcdef"
  instance_type = "t2.micro"
}
```

Full example (including optional):

```
resource "aws_instance" "web" {
  ami                    = "ami-0abcdef"
  instance_type          = "t2.micro"
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web_sg.id]
  key_name               = "keerti-key"

  tags = {
    Name = "web-server"
    Environment = "Dev"
  }
}
```

Understand Resource References

You can **reference other resources** inside a resource block.

Example:

```
vpc_security_group_ids = [aws_security_group.web_sg.id]
```

This tells Terraform to first create the `aws_security_group.web_sg` and then use its ID here.

Terraform automatically builds a **dependency graph**.

> How does Terraform know the order to create resources?
> A: Through dependency graph built using resource references.

Check Attributes Reference

Scroll down the docs to **Attributes Reference**.
It shows what output values are available after creation.

Example (`aws_instance`):

- `arn` → ARN of instance
- `id` → instance ID
- `public_ip` → Public IP
- `private_ip` → Private IP

You can access them:

```
output "public_ip" {
  value = aws_instance.web.public_ip
}
```

NTERVIEW-LEVEL QUESTIONS BASED ON RESOURCE BLOCKS

Question	Explanation
Q1. How do you find correct arguments for a resource?	Always check the Terraform Registry — every resource lists required and optional fields.
Q2. What is the structure of a resource block?	`resource "<provider>_<type>" "<name>" { ... }`
Q3. How do you know which fields are required?	The Registry "Argument Reference" section explicitly marks them as **Required**.
Q4. What is dependency resolution?	Terraform builds a DAG (directed acyclic graph) using references (e.g., `aws_vpc.main.id`).
Q5. Can you mix resources from multiple providers?	Yes, e.g., AWS + Cloudflare + Datadog in same project.
Q6. What is difference between resource and data block?	Resource = create; Data = read existing resource.
Q7. What if a required argument is missing?	`terraform validate` or `plan` will fail with an error.
Q8. Where does Terraform pull the provider plugin from?	Automatically from the Terraform Registry during `terraform init`.
Q9. Can we override provider version?	Yes, using:

1️⃣ Go to [Terraform Registry]
2️⃣ Select your provider → find resource
3️⃣ Read "Example Usage"
4️⃣ Copy required arguments
5️⃣ Add optional arguments (as needed)
6️⃣ Add `tags` always (good AWS practice)
7️⃣ Validate → Plan → Apply
8️⃣ Add `output` to expose attributes